


---

## Master : Artificial Intelligence & Digital Computing

# Rapport TP N°3 : Prompt Engineering

Analyse de sentiment avec Google Gemini

---

**Réalisé par :**
**Fatima Zahra Abbar**

**Encadré par :**
**Pr. L. DAOUDI**

**Année universitaire 2026-2027**

# Table des matières


1 Objectifs du TP . . . . . 2
2 Environnement de travail . . . . . 2
3 Partie 1 : tokenisation et premier appel à Gemini . . . . . 3
4 Step 1 : objectifs et métriques . . . . . 4
5 Step 2 : préparation des données . . . . . 6
6 Step 3 : élaboration des prompts . . . . . 8
7 Step 4 : évaluation des prompts . . . . . 9
8 Analyse . . . . . 11
9 Conclusion . . . . . 11

# 1 Objectifs du TP

Ce TP utilise un grand modèle de langage (LLM) pour classer des avis de films IMDB en **positive** ou **negative**, sans entraîner de modèle. Trois stratégies de prompt sont comparées :

- **Zero-shot** : la consigne seule, sans exemple ;
- **Few-shot** : la consigne et des exemples d'avis annotés ;
- **Chain-of-Thought (CoT)** : le few-shot et des instructions de raisonnement étape par étape.

Les prompts sont évalués avec le score **micro-F1** sur 20 avis de référence (*gold examples*). Le LLM utilisé est **Google Gemini** (`gemini-3.5-flash-lite`), appelé via LangChain, à la place de l'API OpenAI du support de cours.

# 2 Environnement de travail

| **Élément** | **Choix** |
|---|---|
| Éditeur | VS Code + extension Jupyter |
| Gestion du projet | uv (`uv venv`, `uv sync`) |
| LLM | Google Gemini, modèle `gemini-3.5-flash-lite` |
| Accès au modèle | LangChain + `langchain-google-genai` |
| Clé API | Google AI Studio, stockée dans `.env` (non publiée) |
| Bibliothèques | tiktoken, pandas, numpy, scikit-learn, matplotlib, tqdm |

TABLE 1 – Outils utilisés

**Choix du modèle.** Le TP envoie environ 460 requêtes (20 avis × 23 évaluations de prompts). Dans l'offre gratuite, les modèles Flash-Lite ont un quota journalier bien plus large que les modèles Flash. Un limiteur de débit (12 requêtes par minute) et un cache des réponses protègent ce quota.

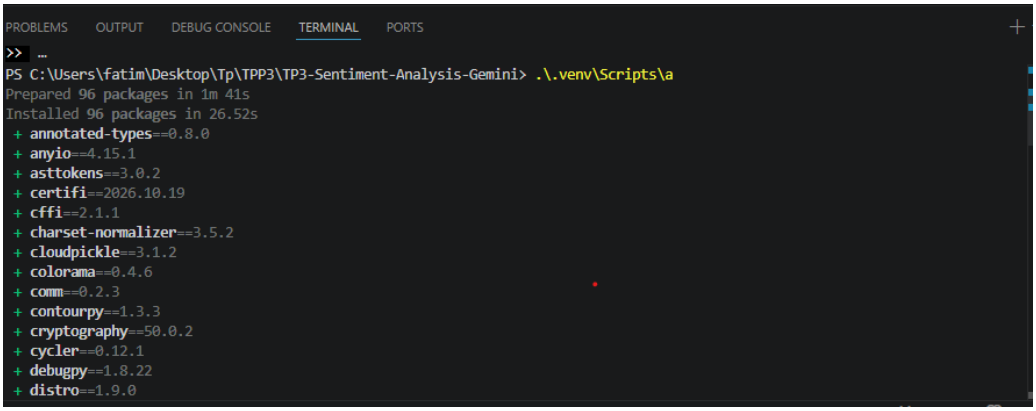



FIGURE 1 – Création de l'environnement virtuel et installation des dépendances avec uv

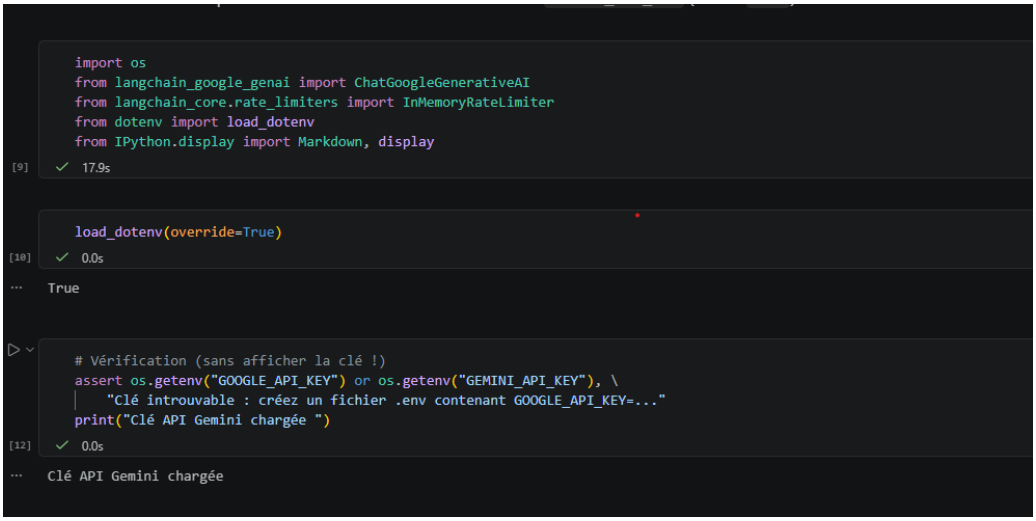



FIGURE 2 – Chargement de la clé API Gemini depuis le fichier `.env`

# 3 Partie 1 : tokenisation et premier appel à Gemini

Un LLM lit des **tokens** (morceaux de mots), pas des mots. Le nombre de tokens détermine le coût d'un appel et la place occupée dans la fenêtre de contexte.

- `tiktoken.encoding_for_model("gpt-4o")` renvoie l'encodage `o200k_base`.
- Le prompt « Vous êtes un expert en marketing digital. C'est quoi un digital marketer ? » est découpé en **16 tokens**.
- Chaque mot courant (avec son espace initiale) forme un seul token ; la ponctuation et l'apostrophe sont isolées.
- Le tokenizer de Gemini compte **17 tokens** pour le même prompt : un nombre de tokens n'est comparable qu'entre modèles du même fournisseur.

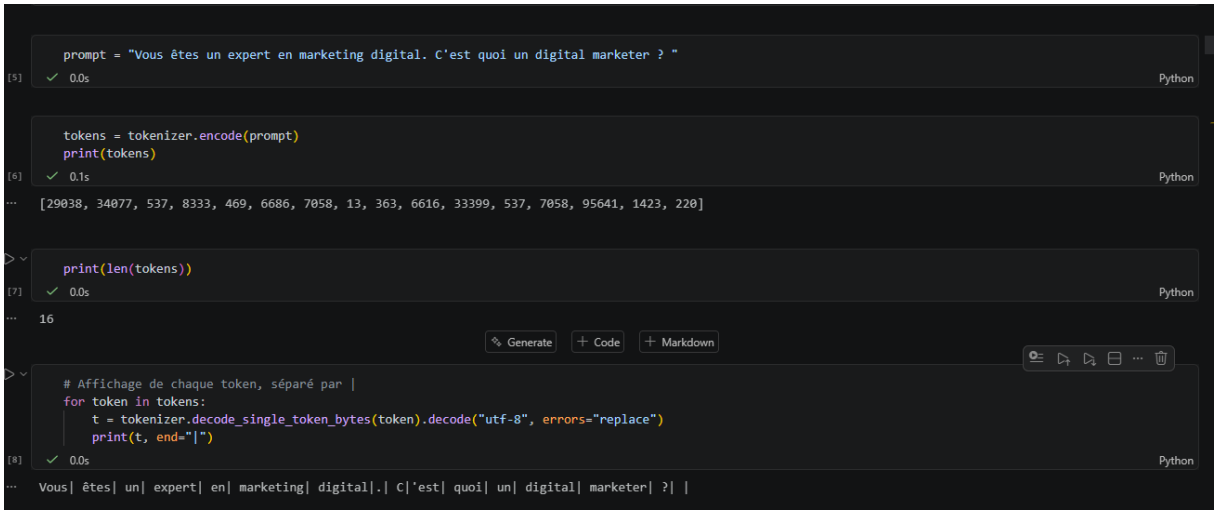



FIGURE 3 – Tokenisation du prompt avec tiktoken

Le premier appel envoie un message *system* (« Vous êtes un expert en marketing digital ») et un message *user* (« C'est quoi un digital marketer ? »). Gemini répond en 28,6 s ; l'appel consomme 16 tokens en entrée et 611 en sortie.

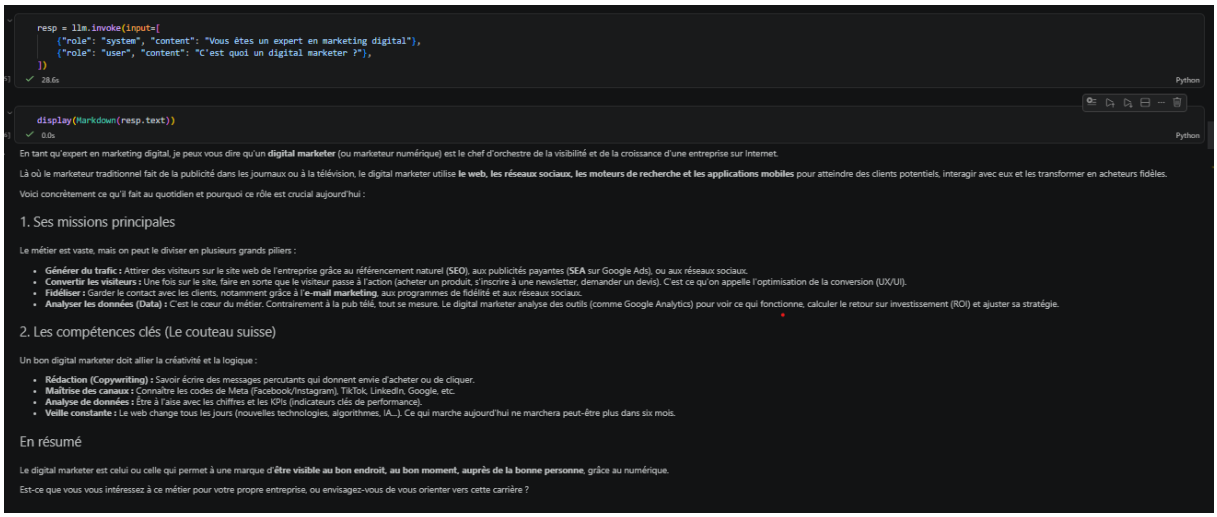



FIGURE 4 – Réponse de Gemini à la question sur le digital marketer

# 4 Step 1 : objectifs et métriques

— **Accuracy** : nombre de prédictions correctes divisé par le nombre total de prédictions.

— **Micro-F1** : TP, FP et FN sont additionnés sur toutes les classes, puis la précision, le rappel et le F1 sont calculés sur ces totaux.

| **Prédit \ Réel** | **Positif** | **Négatif** |
|---|---|---|
| Positif | 50 (TP) | 10 (FP) |
| Négatif | 5 (FN) | 100 (TN) |

TABLE 2 – Matrice de confusion de l'exemple du cours

$$\text{Précision} = \frac{50}{50+10} = 0{,}833 \qquad \text{Rappel} = \frac{50}{50+5} = 0{,}909$$

$$F_1 = \frac{2 \times 0{,}833 \times 0{,}909}{0{,}833 + 0{,}909} = 0{,}870$$

Le calcul refait avec scikit-learn retrouve ces valeurs, et montre que le micro-F1 calculé sur les deux classes vaut **0,909**, comme l'accuracy (150/165). Le 0,870 du cours est donc le F1 de la seule classe positive. En classification mono-étiquette, chaque erreur compte une fois comme FP (classe prédite) et une fois comme FN (vraie classe) : micro-précision, micro-rappel, micro-F1 et accuracy sont égaux.

```python
from sklearn.metrics import f1_score, precision_score, recall_score, accuracy_score

# Reconstruction des 165 prédictions de la matrice de confusion du cours
y_true = ["positive"] * 50 + ["negative"] * 10 + ["positive"] * 5 + ["negative"] * 100
y_pred = ["positive"] * 50 + ["positive"] * 10 + ["negative"] * 5 + ["negative"] * 100

p = precision_score(y_true, y_pred, pos_label="positive")
r = recall_score(y_true, y_pred, pos_label="positive")
print(f"Précision (classe positive) = {p:.3f}")
print(f"Rappel    (classe positive) = {r:.3f}")
print(f"F1        (classe positive) = {f1_score(y_true, y_pred, pos_label='positive'):.3f}   <- valeur du cours (0,870)")
print(f"F1 micro  (toutes classes)  = {f1_score(y_true, y_pred, average='micro'):.3f}")
print(f"Accuracy                    = {accuracy_score(y_true, y_pred):.3f}")
```

✓ 0.0s

```
Précision (classe positive) = 0.833
Rappel    (classe positive) = 0.909
F1        (classe positive) = 0.870   <- valeur du cours (0,870)
F1 micro  (toutes classes)  = 0.909
Accuracy                    = 0.909
```

FIGURE 5 – Vérification de l'exemple du cours avec scikit-learn

# 5 Step 2 : préparation des données

| **Étape** | **Résultat** |
|---|---|
| Corpus `IMDB Dataset.csv` | 50 000 avis, colonnes `review` et `sentiment` |
| Équilibre des classes | 25 000 positive, 25 000 negative |
| Nettoyage | `review` renommée en `text`, balises `<br />` supprimées |
| Colonne `label` | 1 = positive, 0 = negative |
| Split 80/20 (`random_state=42`) | 40 000 exemples few-shot, 10 000 avis de référence |
| Gold examples | 20 avis : 11 negative, 9 positive |
| Sauvegarde | `data/gold_examples.json` |

TABLE 3 – Préparation du jeu de données

— **Deux réservoirs séparés** : on n'évalue jamais un prompt sur des avis qu'il contient déjà.
— `random_state=42` : le tirage est aléatoire mais identique à chaque exécution, donc reproductible.
— **20 avis seulement** : pour limiter le temps d'exécution et la consommation de quota.

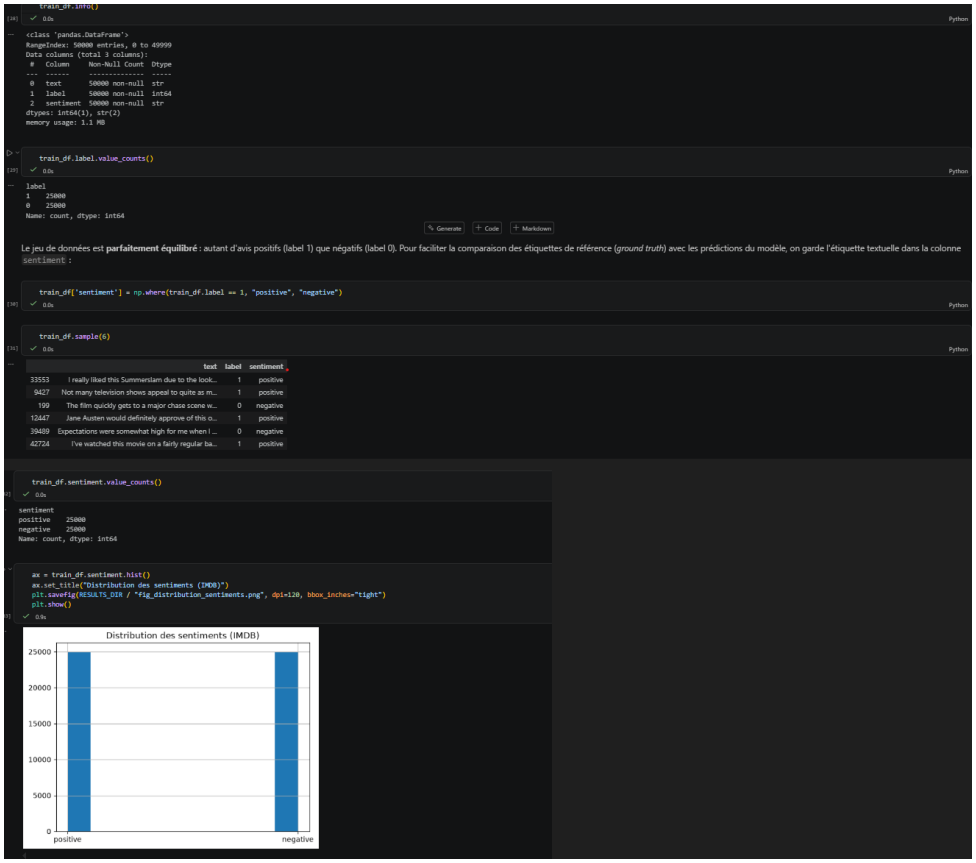



FIGURE 6 – Structure et répartition des sentiments du jeu de données

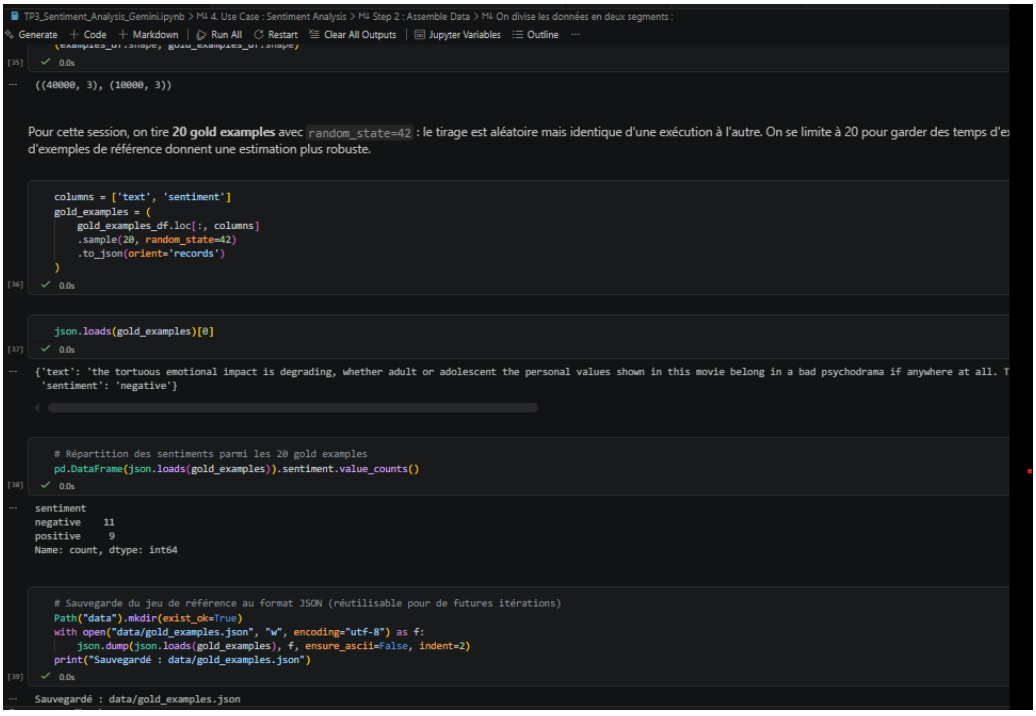



FIGURE 7 – Découpage 80/20 et sélection des 20 gold examples

# 6 Step 3 : élaboration des prompts

Les trois prompts partagent la même consigne :

```
Classify the sentiment of movie reviews presented in the input as '
    positive' or 'negative'
Movie reviews will be delimited by triple backticks in the input.
Answer only 'positive' or 'negative'
Do not explain your answer.
```

| Prompt | Contenu | Tokens |
|---|---|---|
| Zero-shot | Consigne seule | 49 |
| Few-shot | Consigne + 4 exemples équilibrés en ordre aléatoire | 2 014 |
| Chain-of-Thought | Few-shot + instructions : lire attentivement, estimer la probabilité que l'avis soit positif | 2 071 |

TABLE 4 – Les trois prompts (tokens mesurés avec tiktoken sur le premier tirage)

La fonction `create_examples` évite deux biais connus :

— **biais de l'étiquette majoritaire** : elle tire autant d'avis positifs que négatifs ;

— **biais de récence** : elle mélange les exemples, donc aucune classe n'est toujours en fin de prompt.

La fonction `create_prompt` assemble le message système puis, pour chaque exemple, un message *user* (l'avis) suivi d'un message *assistant* (le sentiment attendu).

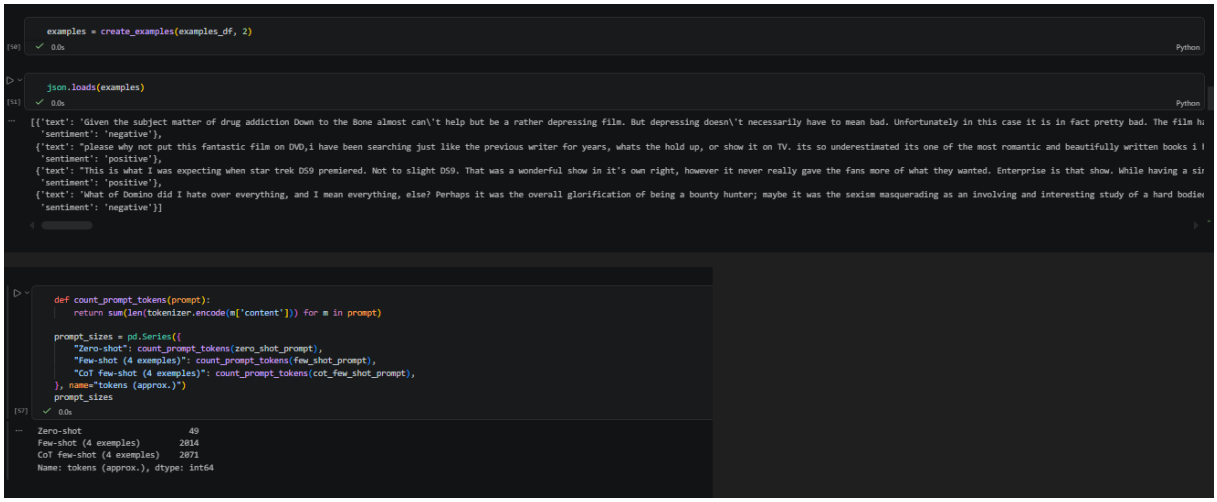



FIGURE 8 – Exemples équilibrés et taille des prompts

# 7 Step 4 : évaluation des prompts

La fonction `evaluate_prompt` envoie chaque gold example à Gemini, convertit la réponse en étiquette et renvoie `f1_score(..., average="micro")`.

**Adaptations pour Gemini :**

- la réponse est lue avec `response.text`, car `response.content` peut être une liste avec Gemini 3 ;
- `temperature=0` n'est pas transmis : gemini-3.5-flash-lite impose son échantillonnage ;
- un cache disque évite de renvoyer une requête déjà faite.

**Correction du code du cours :** dans la boucle des 10 tirages, l'instruction suivante enregistrait le score few-shot à la place du CoT :

```
cot_few_shot_performance.append(few_shot_micro_f1)
```

Le notebook évalue réellement les deux prompts à chaque tirage.

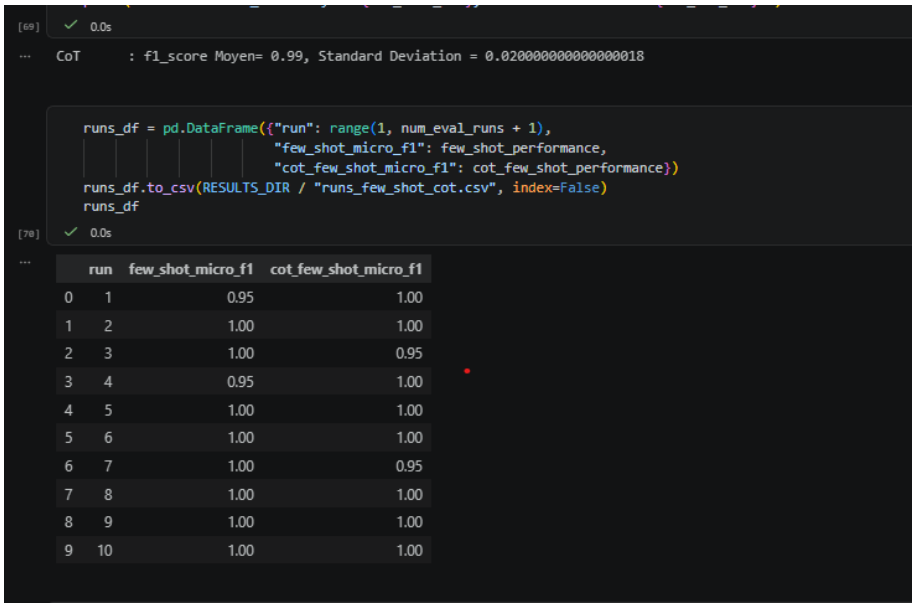
| Prompt | Micro-F1 (1 tirage) | Moyenne (10 tirages) | Écart-type |
|---|---|---|---|
| Zero-shot | 1,00 (20/20) | – | – |
| Few-shot | 1,00 (20/20) | 0,99 | 0,02 |
| Chain-of-Thought | 0,95 (19/20) | 0,99 | 0,02 |

TABLE 5 – Résultats avec gemini-3.5-flash-lite sur 20 gold examples

- **Matrices de confusion** : le zero-shot et le few-shot classent correctement les 11 avis négatifs et les 9 positifs ; l'unique erreur du CoT est un avis négatif prédit positif.
- **10 tirages** : le few-shot obtient 0,95 aux tirages 1 et 4, le CoT aux tirages 3 et 7, et 1,00 partout ailleurs.
- **Durée** : la boucle a pris 52 minutes (400 requêtes).

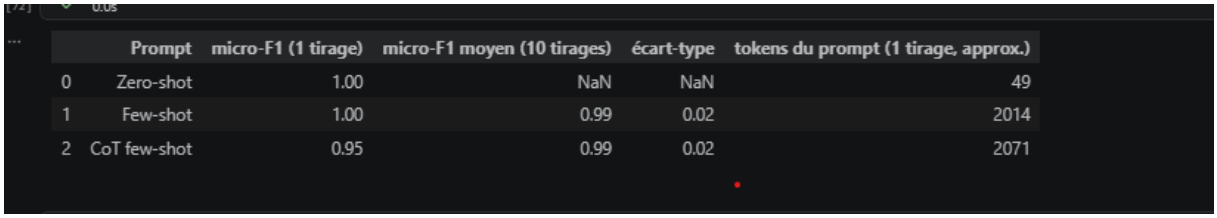



FIGURE 9 – Micro-F1 des trois prompts et matrices de confusion




| | run | few_shot_micro_f1 | cot_few_shot_micro_f1 |
|---|---|---|---|
| 0 | 1 | 0.95 | 1.00 |
| 1 | 2 | 1.00 | 1.00 |
| 2 | 3 | 1.00 | 0.95 |
| 3 | 4 | 0.95 | 1.00 |
| 4 | 5 | 1.00 | 1.00 |
| 5 | 6 | 1.00 | 1.00 |
| 6 | 7 | 1.00 | 0.95 |
| 7 | 8 | 1.00 | 1.00 |
| 8 | 9 | 1.00 | 1.00 |
| 9 | 10 | 1.00 | 1.00 |

FIGURE 10 – Variabilité sur 10 tirages d'exemples



| | Prompt | micro-F1 (1 tirage) | micro-F1 moyen (10 tirages) | écart-type | tokens du prompt (1 tirage, approx.) |
|---|---|---|---|---|---|
| 0 | Zero-shot | 1.00 | NaN | NaN | 49 |
| 1 | Few-shot | 1.00 | 0.99 | 0.02 | 2014 |
| 2 | CoT few-shot | 0.95 | 0.99 | 0.02 | 2071 |

FIGURE 11 – Tableau récapitulatif des résultats

# 8 Analyse

**Pourquoi le micro-F1 plutôt que l'accuracy ?** Ici les deux sont égaux, car la tâche est mono-étiquette. Le micro-F1 reste le cadre général : il s'étend aux tâches multi-classes et une réponse `unknown` y compte comme une erreur.

**Quel prompt est le plus performant ?** Les trois sont pratiquement à égalité. Le zero-shot atteint 1,00 sans aucun exemple ; le few-shot et le CoT obtiennent 0,99 en moyenne. L'analyse de sentiment sur IMDB est une tâche facile pour un LLM récent : la consigne seule suffit.

**Pourquoi répéter l'évaluation 10 fois ?** Sur le premier tirage, le CoT (0,95) semble moins bon que le few-shot (1,00). Sur 10 tirages, les deux obtiennent 0,99 ± 0,02. Sans répétition, on aurait conclu à tort que le CoT dégrade la performance.

**Le Chain-of-Thought apporte-t-il quelque chose ?** Non, sur ce modèle. Le prompt CoT interdit d'expliquer la réponse, donc le raisonnement n'est jamais écrit, et les modèles Gemini 3 raisonnent déjà en interne.

**Quel est le coût de chaque prompt ?** Le few-shot compte 2 014 tokens contre 49 pour le zero-shot, soit environ 41 fois plus, sans gain de performance.

**Limites.** Avec 20 gold examples, chaque erreur coûte 0,05 point. Le corpus contient 418 avis en double, et l'échantillonnage imposé par Gemini peut faire varier légèrement deux appels identiques.

# 9 Conclusion

Ce TP montre qu'un LLM classe des avis de films sans entraînement, par simple ingénierie de prompt, et qu'un protocole rigoureux est indispensable pour comparer des prompts. Avec gemini-3.5-flash-lite, le **zero-shot** offre le meilleur compromis : micro-F1 de 1,00, aucune variabilité liée aux exemples et un prompt environ 41 fois plus court. Pour départager réellement les prompts, il faudrait plusieurs centaines d'avis de référence, ou des avis plus difficiles (ironiques, nuancés).